RSN

Application Testing — Implementation Proposal & Plan

RSN Platform · Structured coverage, traceability, and continuous QA

28 May 2026 · Version 1.0

This document sets out how RSN will implement the proposed structured application-testing approach — turning testing from a one-off feedback exercise into a **repeatable, traceable QA process** in which every user-reported issue is tied to a functional component and to the technical module that owns it. It adopts the proposed four-step framework in full and maps each step to a concrete RSN deliverable. It is a standing capability intended to apply to every test cycle and event, not a response to any single session.

CONTEXT

- This plan adopts the four-step testing framework in full; each deliverable below maps one-to-one to a step in that framework.
- The objective is thorough, repeatable coverage of the platform, with every issue traceable from *user symptom → functional component → technical module → fix → verification*.
- The approach is primarily structured, moderated **human testing**. AI is applied only where it removes manual effort — classifying incoming feedback, flagging duplicates, and drafting the symptom-to-module mapping for human review.
- It is designed to stay lightweight: a single living component map and issue register, not a heavyweight toolchain — so it does not "fall into disuse."
- The framework is generic by design: it applies to every test cycle and event as a standing capability, independent of any individual session's findings.
- This is the implementation plan; the artifacts it describes — component map, dependency map, and issue register — will be maintained as living documents.

WHAT RSN WILL DELIVER

#	What	Example	Why
1	**Build a complete component / functional-area map of the RSN platform, at module granularity, with each area linked to the code path that owns it.**	**Public user:** Authentication & onboarding; Profile; Home / dashboard; Invites & join-requests; Main networking room (lobby); Matching & round flow; Breakout / conversation room; In-event chat & DM; Rating; Notifications; Billing. **Internal / admin:** User management; Event & session moderation; Host control center; Co-host management; Reports & blocks; Join-request approval.	Gives everyone one shared map of the application to test against and to track all feedback — the foundation the remaining steps build on.

#	What	Example	Why
2	**Document the upstream / downstream relationships between components as a dependency and data-flow map.**	Host / admin moderation console → matching engine → main-room presence → breakout room. Auth → socket connection → presence heartbeat → matching eligibility → room assignment. Delivered as a flow diagram plus a supporting table.	Lets an issue seen in one surface (e.g. "alone in a breakout room") be traced to its true cause upstream (e.g. presence or matching state), rather than being treated as an isolated UI bug.
3	**Stand up a component-keyed issue register and tie all test feedback to it.**	A single structured register (shared sheet / board) where every issue is logged against a component from Row 1, with severity, status, a duplicate tally, and a link to the owning technical module.	Everyone can see all issues for a component in one place; duplicates get a tally rather than a re-entry; redundancy drops and coverage becomes visible.
4	**Make the record usable, maintainable, and traceable end-to-end through to engineering.**	Each entry chains *user-facing issue* → *component* → *technical module* → *fix status* → *verification*, and is reviewed every test cycle. Kept deliberately lightweight — one living register, not a clunky toolchain.	Provides traceability from a user symptom to the exact code fix, supports coverage sign-off, and funnels issues to engineering already tied to the responsible component.

WORKED EXAMPLE — HOW A SINGLE OBSERVATION FLOWS THROUGH THE FRAMEWORK

The framework's Steps 1–3 applied to a single observation. Each row reads **symptom → component → upstream cause → owning module**. *The examples below are illustrative only — included to demonstrate intent, not actual findings or recommendations.*

User-facing observation (illustrative)	**Component (Step 1)**	**Possible upstream cause (Step 2)**	*Owning module*
A participant's video tile shows blank for others	Main networking room — video	The media track fails to subscribe when the participant joins	*Video / track subscription*
Two users see a different participant count on screen	Main networking room — presence	Presence updates are applied as deltas with no periodic reconciliation	*Presence / sync layer*
A breakout timer shows a different value to each person	Breakout / conversation room — timer	The countdown runs locally instead of from a shared end-time	*Timer / round service*
An invite link opens an error page	Invites & join-requests	An expired or malformed token is not handled gracefully	*Invite service*
A user cannot change their profile photo on mobile	Profile	The upload control sits below the fold on small screens	*Profile page (client)*

Examples are illustrative only and demonstrate how an observation maps through the framework; in practice the maintained register links each real entry to the precise file and location for engineering.

PHASED PLAN

Phase 1 — Component map

Produce the functional-area inventory (Row 1), reviewed and agreed. **Done when:** every public and internal surface is listed and has a clear owner.

Phase 2 — Dependency & data-flow map

Produce the relationship map (Row 2) as a diagram plus supporting table. **Done when:** each component's upstream and downstream links are recorded.

Phase 3 — Issue register

Stand up the component-keyed register (Row 3), seeded with results from the most recent test session. **Done when:** every logged issue is recorded against a component, with severity and status.

Phase 4 — Traceability & AI assist

Add the issue → module → fix-status chaining (Row 4) and AI-assisted classification and duplicate detection. **Done when:** an issue can be followed from symptom to fix to verification.

Phase 5 — Continuous cadence

Each moderated test session logs against the map in real time; afterwards the register is updated, deduped, and funneled to engineering, and coverage gaps feed the next session. **Done when:** the loop runs unaided for each event.

USAGE

- During testing, whoever is moderating knows exactly which component each piece of feedback belongs to.
- Testers record results against the relevant component as they go.
- App owners can confirm every part of the platform has been tested and review results per component.
- Issues reach engineering already tied to the technical component responsible, ready for a fix.
- AI is applied wherever it reduces effort — classifying feedback, flagging duplicates, and drafting the symptom-to-module mapping for human review.